

Project Fusion

A Unified Orchestration Architecture for
Regulated Digital finance

Document Classification: Technical Whitepaper (Prototype Submission)

Prepared By : Praveenkumar Premkumar [B.Eng, Master of Science]

Project Fusion.....	1
1. Executive Summary.....	3
2. Market Context & Problem Statement.....	4
2.1 The Fragmentation of Value Transfer.....	4
2.2 The "Split-Brain" Infrastructure Risk.....	4
2.3 Regulatory Arbitrage and Compliance Gaps.....	4
3. Design Philosophy & Core Principles.....	5
3.1 Separation of Concerns (Orchestration vs. Settlement).....	5
3.2 Non-Custodial Architecture.....	5
3.3 The Singleness of Money.....	5
3.4 Deterministic State Management.....	5
4. System Architecture.....	5
4.1 High-Level Component View.....	5
4.2 The Fusion Control Plane.....	6
4.3 The Adapter Layer.....	6
4.4 The Governance Layer.....	6
5. The Persistent State Machine (Core Protocol).....	7
5.1 The "Ghost Money" Prevention Mechanism.....	7
5.2 Lifecycle States and Transition Logic.....	7
6. Data Models & API Specifications.....	8
6.1 Instruction Schema (JSON).....	8
6.2 Policy Permit Schema.....	9
6.3 Governance Proof Structure.....	9
7. Policy & Governance Framework.....	10
7.1 Policy-First Execution.....	10
7.2 Open Policy Agent (OPA) Integration.....	10
7.3 The Regulator Observer Node.....	10
8. Security Architecture & Hardening.....	10
8.1 Identity & Access Management (mTLS).....	10
8.2 Cryptographic Standards.....	11
8.3 Data Privacy & Zero-Knowledge Proofs (Future Roadmap).....	11
9. Regulatory Compliance Mapping.....	11
9.1 MAS TRM Guidelines Alignment.....	11
9.2 FATF Travel Rule Compliance.....	11
10. Detailed Use Cases.....	11
10.1 Purpose-Bound Money (PBM) Disbursements.....	11
10.2 Cross-Border PvP Settlement (Fiat-to-Crypto).....	12
11. Prototype Constraints — Must Be Addressed Before Production.....	12
11.1 Authentication and Signing.....	12

11.2 Custody Integration.....	12
11.3 Governance Infrastructure.....	12
11.4 Testing and Validation.....	13
12. Conclusion.....	13
REFERENCES.....	14

1. Executive Summary

Financial innovation is currently hamstrung by a structural paradox: while execution rails—such as SWIFT for payments, MPC wallets for crypto, and FIX for securities—have matured significantly, the orchestration layer that governs them remains dangerously fragmented. Financial institutions today operate "islands of automation," maintaining disparate systems for distinct asset classes. This segregation leads to duplicated compliance efforts, operational inefficiency, and systemic synchronization risks.

Project Fusion defines a unified **Control Plane** for financial orchestration. It is designed not as a settlement rail or custodian, but as a **Deterministic State Machine** that sits above existing infrastructure to coordinate the lifecycle of financial instructions.

By decoupling the **Intent of Value Movement** from the **Mechanism of Settlement**, the Project Fusion architecture achieves three critical objectives. First, it establishes **Unified Governance** by specifying a single Policy Engine (OPA) to apply AML/CFT rules uniformly across Fiat, Crypto, and Securities. Second, it provides **Technical Atomicity** through a "Check-Lock-Execute" protocol; this architecture prevents "Ghost Money" (funds moving without internal records) via persistent state management and asynchronous reconciliation. Finally, it creates a mechanism for **Regulatory Observability** via a dedicated Observer Node interface, allowing regulators to audit compliance mathematically.

This whitepaper serves as the definitive technical specification for the Project Fusion prototype architecture, detailing the data models, failure semantics, and security controls defined for the sandbox environment.

2. Market Context & Problem Statement

2.1 The Fragmentation of Value Transfer

Modern financial institutions do not lack connectivity; they lack coherence. A typical Tier-1 bank operates a complex web of disconnected infrastructure. Legacy Core Banking systems handle domestic payments (FAST/PayNow) and SWIFT, while isolated Digital Asset Sandboxes manage tokenized bonds or custody on separate cloud stacks. Simultaneously, Wealth Management Systems utilize proprietary order management systems (OMS) for equities.

These systems do not share a common state. A "Programmable Money" (PBM) script running on a blockchain has no visibility into the bank's core ledger. This necessitates complex, brittle middleware to bridge the gap, often relying on "fire-and-forget" API calls that lack transactional integrity.

2.2 The "Split-Brain" Infrastructure Risk

The most critical technical risk in hybrid finance is the **Synchronization Gap**. Consider a

scenario where a smart contract authorizes a payout and the middleware instructs the bank to send fiat. If the bank sends the fiat but the API response times out, the smart contract erroneously assumes the payment failed. It might retry (causing a double spend) or revert its state. Meanwhile, the bank ledger shows the funds have departed. The consequence is "Ghost Money"- a dangerous discrepancy between the programmable state and the legal settlement state.

2.3 Regulatory Arbitrage and Compliance Gaps

Currently, compliance logic is often hardcoded into execution silos. SWIFT payments trigger specific Sanctions Screening tools, while crypto withdrawals trigger separate Chain Analysis tools. Internal transfers might bypass both checks entirely. This inconsistency creates **Regulatory Arbitrage** within the institution itself. Project Fusion posits that policy serves as a horizontal enforcement layer, distinct from vertical execution silos.

3. Design Philosophy & Core Principles

3.1 Separation of Concerns (Orchestration vs. Settlement)

The Fusion architecture is defined by the principle that **Orchestration** (deciding *if* and *where* to move value) is distinct from **Execution** (the act of moving value). The architecture assigns Fusion responsibility for the validation of intent, policy checking, routing, and state recording. Conversely, external rails retain responsibility for custody, liquidity management, and settlement finality.

3.2 Non-Custodial Architecture

Project Fusion is explicitly defined as a non-custodial control plane. At no point does the platform take legal possession of underlying assets, private keys, or fiat liquidity. All assets remain under the control of sponsor banks, licensed qualified custodians, or regulated brokerage partners. The Fusion Ledger records shadow balances for the purpose of state coordination and double-entry integrity, but these records do not constitute a legal deposit.

3.3 The Singleness of Money

Regardless of the form factor Commercial Bank Money, Central Bank Digital Currency (CBDC), or Stablecoin the representation of value must remain singular. The design specifies a **Shadow Ledger** (Double-Entry) that strictly mirrors external movements. The architecture requires that no external movement be authorized without a corresponding lock and debit in the Shadow Ledger.

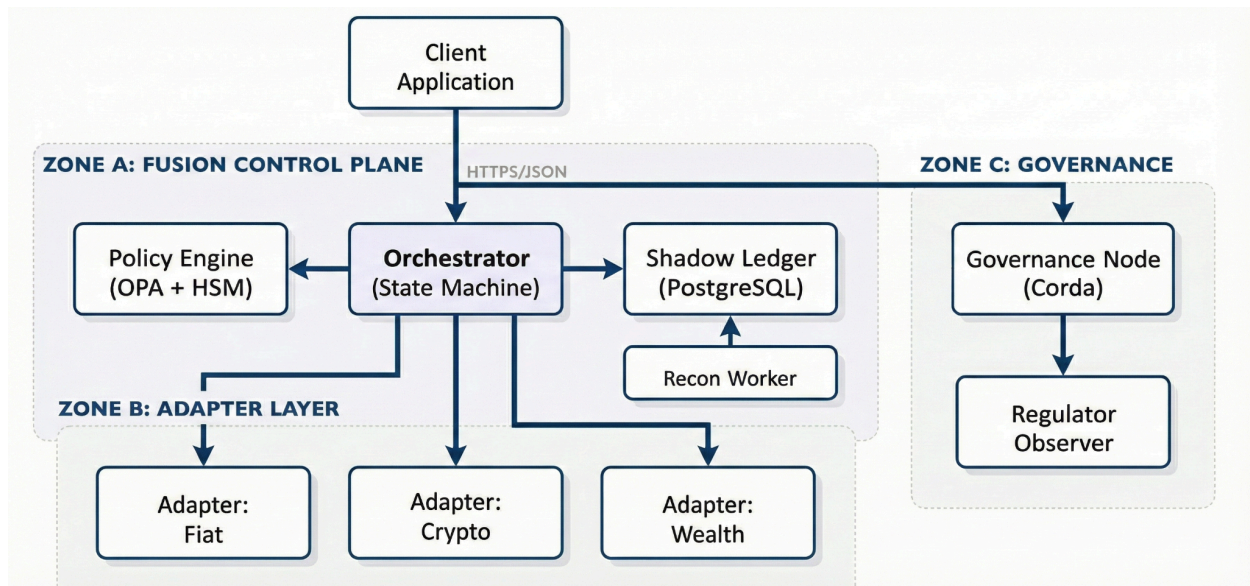
3.4 Deterministic State Management

Distributed systems cannot rely on "Happy Path" assumptions. The architecture assumes network partitions, API timeouts, and database locks are inevitable. Therefore, the system is

specified as a **Persistent State Machine**. Every transition is defined to be recorded to disk before any external action is taken, ensuring the system can recover its exact state even after a catastrophic power loss or crash.

4. System Architecture

4.1 High-Level Component View



Flowchart 1 : System Architecture Diagram

4.2 The Fusion Control Plane

The Control Plane is the core of the system, designed as a set of microservices deployed in a secure VPC (Virtual Private Cloud). The **Orchestrator** implements the Saga pattern and State Machine logic; it is defined as stateless to allow horizontal scaling, offloading state persistence to the database. The **Policy Engine** runs as an isolated Open Policy Agent (OPA) service, accepting JSON instructions and returning a cryptographically signed **Policy Permit**-a verifiable authorization artifact rather than a simple boolean decision. The **Shadow Ledger** is defined as a high-performance SQL database (PostgreSQL) optimized for ACID transactions, recording the "Internal Truth" of balances.

4.3 The Adapter Layer

To maintain a clean core, all rail-specific complexity is delegated to the Adapter Layer. Its primary responsibility is defined as translating generic Fusion Instructions into protocol-specific

messages (e.g., converting a generic send_payment into an MT103 text file). Furthermore, the architecture specifies that all adapters implement idempotency keys when calling external providers to prevent double-execution during retries or network interruptions.

4.4 The Governance Layer

A permissioned DLT (Distributed Ledger Technology) node, such as R3 Corda or Hyperledger Fabric, acts as the immutable audit log. The architecture defines that critical state transitions specifically LOCKED and SETTLED states-be hashed and anchored to this ledger. To preserve privacy and prevent data leakage, the specification restricts storage on the Governance Ledger to **cryptographic hashes and Zero-Knowledge Proofs (ZKPs)** only. No PII, account balances, or transaction amounts are stored on the shared ledger.

5. The Persistent State Machine (Core Protocol)

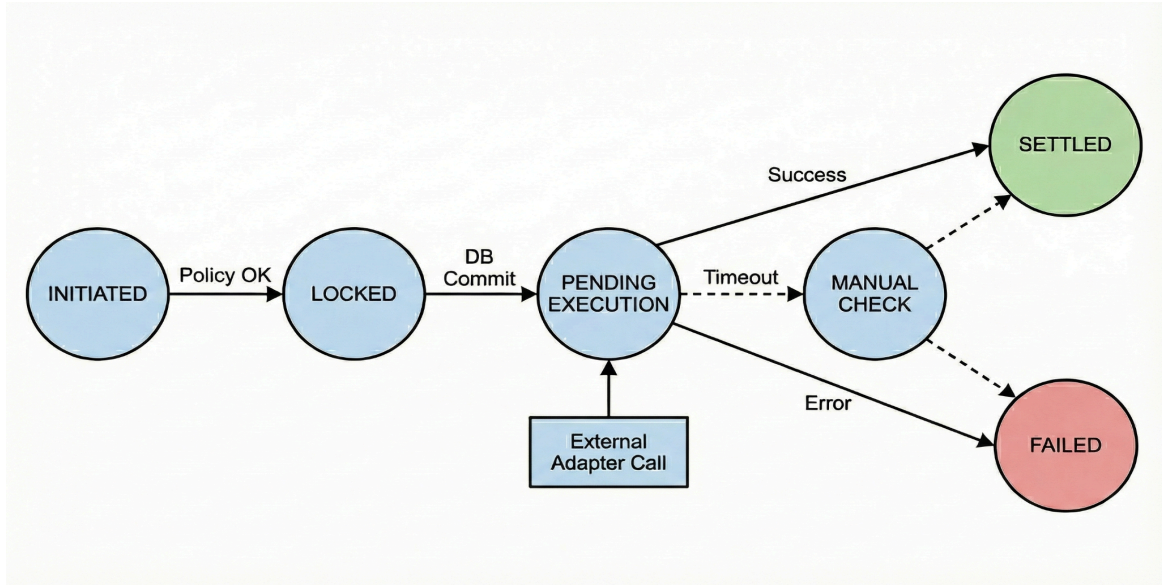
5.1 The "Ghost Money" Prevention Mechanism

Traditional systems often fail because they wrap external API calls in database transactions. The Fusion architecture explicitly excludes this pattern. The "Ghost Money" fix utilizes the **"Pending Execution"** pattern, ensuring that the intent to execute is persisted before the execution is attempted.

5.2 Lifecycle States and Transition Logic

The architecture defines the instruction lifecycle through the following states:

1. **INITIATED**: Upon receipt, the system validates the JSON schema and authenticates the client using configured mechanisms (e.g., mTLS).
2. **LOCKED (The Compliance Gate)**: The Policy Engine evaluates rules against the instruction. Simultaneously, the Shadow Ledger debits the sender's shadow balance to reserve funds. If this step fails, the instruction is rejected instantly.
3. **PENDING_EXECUTION (The Safety Anchor)**: The system commits a record to the database explicitly stating: *"I am about to call Adapter X."* This commit is defined to occur before the network request is initiated.
4. **SETTLED / FAILED**: Based on the response from the Adapter, the database is updated to reflect the final outcome.
5. **MANUAL_CHECK (The Safety Net)**: If the Adapter times out or the server crashes while in PENDING_EXECUTION, the system reboots into this state. The architecture precludes auto-rollback, instead specifying a reconciliation process to determine the true state of the external rail.



Flowchart 2 : Instruction Lifecycle

5.3 Asynchronous Reconciliation Algorithms

A background worker is defined to run at configured intervals (e.g., 60 seconds) to scan for "Stuck" transactions. The reconciliation logic operates as follows:

1. Query the database for instructions where state is `PENDING_EXECUTION` and the timestamp exceeds the timeout threshold.
2. For each identified instruction, query the specific Adapter for the transaction status.
3. If the status is **COMPLETED**, update the local state to `SETTLED` and log the recovery.
4. If the status is **FAILED**, update the local state to `FAILED` and trigger a compensation event (refund).
5. If the status is **UNKNOWN**, generate a critical alert for manual intervention.

6. Data Models & API Specifications

To ensure interoperability, Fusion defines strict JSON schemas.

6.1 Instruction Schema (JSON)

The core message passed from Client to Orchestrator.

Listing 1: Instruction Schema

```
{  
  "amount": "2500.00",  
  "currency": "SGD",  
  "sender": "Alice_Corp",  
  "recipient": "Bob_Supply",  
  "purpose": "TRANSFER"  
}
```

6.2 Policy Permit Schema

The artifact produced by the Policy Engine (OPA) and signed by the HSM

Listing 2: Policy Permit Schema

```
{  
  "permit_id": "perm-uuid-v4",  
  "instruction_id": "uuid-v4-string",  
  "decision": "PERMIT",  
  "evaluated_rules": [  
    "Check_AML_Threshold_Passed",  
    "Check_Destination_Sanctions_Passed",  
    "Check_PBM_Purpose_Match"  
  ],  
  "constraints": {  
    "valid_until": "2026-01-03T12:05:00Z",  
    "max_retry_count": 3  
  },  
  "signature": "hsm-signature-string-ecdsa-sha256"  
}
```

6.3 Governance Proof Structure

The hashed record stored on the Corda/Hyperledger node.

Listing 3: Governance Proof Structure

```
{
  "proof_id": "gov-uuid",
  "event_type": "SETTLEMENT_FINALIZED",
  "instruction_hash": "sha256-hash-of-original-instruction",
  "permit_hash": "sha256-hash-of-permit",
  "adapter_proof": "external-rail-tx-id",
  "timestamp": "2026-01-03T12:01:00Z"
}
```

7. Policy & Governance Framework

7.1 Policy-First Execution

In traditional systems, policy checks are often treated as advisory. In Fusion, the architecture defines them as mandatory "gates." The Orchestrator logic is designed to preclude the construction of an Adapter call unless it possesses a valid, cryptographically signed Policy Permit, ensuring compliance by design.

7.2 Open Policy Agent (OPA) Integration

The architecture specifies the use of OPA (Open Policy Agent) to externalize business logic, allowing policy changes without requiring code deployments. Rules are written in Rego and deployed via a Sidecar pattern or isolated service. For example, a rule might strictly state: *"Deny payment if Currency is PBM_TOKEN and Merchant Category Code is NOT 5411 (Grocery)."*

7.3 The Regulator Observer Node

The Observer Node is defined as a specialized API endpoint restricted to regulatory bodies (e.g., MAS). It provides a sanitized feed of the Governance Ledger. To preserve privacy, the Observer is designed to see that a transaction occurred and that it passed policy, but it does not view the names of the parties involved unless a specific warrant enables the unlocking of that data. Access to this node is strictly read-only and auditable; all inquiries are pre-approved, immutable, and fully logged to ensure non-repudiation of regulatory oversight activities.

8. Security Architecture & Hardening

8.1 Identity & Access Management (mTLS)

The production architecture is defined to enforce a Zero Trust security model. The specification requires that all internal traffic between Fusion components (e.g., Orchestrator to Adapter) be secured via Mutual TLS (mTLS) using internal CA-issued certificates. External client traffic is specified to use HTTPS with TLS 1.3 standards.

8.2 Cryptographic Standards

The architecture defines adherence to specific cryptographic protocols. Signing operations are specified to utilize ECDSA (secp256r1) or Ed25519 for high-performance Permit signing, while SHA-256 is used for ledger integrity and governance proofs. The security model specifies that keys are never to be stored in software or environment variables. Production deployment mandates the use of FIPS 140-2 Level 3 HSMs (e.g., AWS CloudHSM, Thales Luna) for all key storage and signing operations.

8.3 Data Privacy & Zero-Knowledge Proofs (Future Roadmap)

While the current prototype relies on hashing for privacy, the architecture roadmap allows for the integration of ZK-SNARKs. This capability would enable the system to prove compliance statements (e.g., "Sender is over 18") without revealing the underlying data (date of birth).

9. Regulatory Compliance Mapping

9.1 MAS TRM Guidelines Alignment

The architecture is designed to align with MAS Technology Risk Management (TRM) Guidelines. The Policy Engine is configured to enforce pre-transaction risk limits (TRM Section 3), while the specification for mTLS and Role-Based Access Control (RBAC) addresses Access Control requirements (TRM Section 6). Additionally, the stateless design of the Orchestrator supports Active-Active High Availability across Availability Zones, enabling alignment with Availability standards (TRM Section 8).

9.2 FATF Travel Rule Compliance

The data model supports the exchange of originator and beneficiary information required by global regulators. The metadata field in the Instruction Schema is designed to be compliant with IVMS101 data standards, supporting VASP Travel Rule compliance.

10. Detailed Use Cases

10.1 Purpose-Bound Money (PBM) Disbursements

Objective: A government distributes "Education Vouchers" that can only be spent at registered schools.

Process Flow: The user scans a QR code at a Merchant, and Fusion receives an Instruction containing `metadata.sku_list`. The Policy engine checks `thesku_list` against the `ALLOWED_EDUCATION_ITEMS` database. If the merchant is a Bookstore, the policy approves, a permit is issued, and the Adapter moves funds. However, if the merchant is a Clothing Store, the policy denies, no permit is issued, and the Orchestrator returns a 403 Forbidden error. The funds are defined to remain in the wallet.

10.2 Cross-Border PvP Settlement (Fiat-to-Crypto)

Objective: Bank A buys USDC using SGD.

Process Flow: Fusion first locks SGD in Bank A's Shadow Ledger. It then triggers the Crypto Adapter to mint USDC to Bank A's wallet. The system waits for Blockchain Confirmation, handling the Atomicity Gap inherent in cross-chain transactions. If the Blockchain confirms, Fusion commits the SGD deduction. If the Blockchain reverts or fails, Fusion is designed to trigger a "Compensation" event to unlock the SGD in the Shadow Ledger.

11. Prototype Constraints — Must Be Addressed Before Production

This technical whitepaper defines a specific boundary between the current prototype implementation and the mandatory requirements for production deployment. The following constraints represent prototype-specific implementation choices; the corresponding production controls are mandatory requirements for live deployment.

11.1 Authentication and Signing

The prototype utilizes API Key authentication for inter-service communication and HMAC software-based signing for Policy Permits. This is sufficient for functional validation only. Production deployment requires the implementation of Mutual TLS (mTLS) for all service-to-service links and the utilization of FIPS 140-2 Level 3 Hardware Security Modules (HSM) or Multi-Party Computation (MPC) clusters for cryptographic signing.

11.2 Custody Integration

The prototype uses simulated adapters to represent banks and custodians. Production

deployment requires formal legal integration with Sponsor Banks and Qualified Custodians. The system is defined to operate in a non-custodial capacity; therefore, indemnity agreements and operational runbooks are mandatory requirements for establishment with custody partners prior to pilot launch.

11.3 Governance Infrastructure

The prototype simulates the Governance Node using a centralized mock service. Production deployment requires the provisioning of independent nodes owned and operated by distinct consortium members (e.g., Regulator, Bank, Platform Operator) to ensure true decentralized auditability.

11.4 Testing and Validation

Before moving to a production environment, the platform must undergo KPI-driven stress testing, specifically targeting the reconciliation algorithms under simulated network partition events. A formal third-party security audit and penetration test are mandatory prerequisites for the production release candidate.

12. Conclusion

Project Fusion represents a shift from **siload execution** to **unified orchestration**. By accepting the reality of distributed system failures and architecting a **State Machine** capable of handling them, Fusion provides a robust, compliant, and auditable layer for the next generation of digital finance.

The architecture described herein moves beyond the "Happy Path" fallacies of early fintech prototypes, offering a hardened, regulator-ready platform design that respects the singleness of money and the necessity of proactive compliance.

REFERENCES

- [1] Monetary Authority of Singapore (MAS). (2016). FinTech Regulatory Sandbox Guidelines.
- [2] ISO 20022. (2023). Universal Financial Industry Message Scheme.
- [3] SWIFT. (2021). SWIFT gpi: Global Payments Innovation Technical Specifications.
- [4] National Institute of Standards and Technology (NIST). (2017). Digital Identity Guidelines (SP 800-63).
- [5] Bank for International Settlements (BIS). (2020). Project Helvetia: Settling Tokenised Assets in Central Bank Money.
- [6] Open Policy Agent (OPA). (2024). Rego Policy Language Documentation.
- [7] R3. (2023). Corda 5 Technical Whitepaper: Next Gen DLT.
- [8] Enterprise Ethereum Alliance. (2022). Cross-Chain Interoperability Guidelines.
- [9] Stripe. (2023). Designing Resilient Payment Systems: Idempotency keys and Sagas.
- [10] Uber Engineering. (2020). Fulfillment Platform: Architecture for High-Reliability Transactions.
- [11] Gartner. (2023). Market Guide for Financial Orchestration Platforms.
- [12] Financial Action Task Force (FATF). (2021). Updated Guidance for a Risk-Based Approach to Virtual Assets and VASPs.
- [13] Microsoft. (2022). Azure Architecture Guide: Sagas on Azure.
- [14] Confluent. (2023). Transactional Outbox Pattern with Kafka.
- [15] PingCAP. (2025). Architecting FinTech Resilience with Distributed SQL.